Project A: Apartment Management System

Case Scenario and Scope Specification

Module	Software Architecture and Process Models
Student Level	Level 2
Assignment Duration	7 weeks
Document	02 of 07
Assigned Groups	Groups 1, 2, 3, and 4
Product Type	Integrated web-based functional prototype

1. Case Scenario

A medium-sized apartment complex manages multiple buildings, residential units, owners, tenants, management staff, technicians, shared facilities, monthly charges, maintenance work, visitor records, and community announcements. Current activities depend on spreadsheets, paper forms, messaging applications, and disconnected records. This results in duplicated data, delayed maintenance, unclear payment status, inconsistent communication, and limited visibility for residents and management.

The project must deliver an integrated Apartment Management System that provides role-based digital services while preserving clear microservice boundaries. The system should support residents and management staff through a shared React application and Spring Boot services behind an API Gateway.

2. Project Goal

> **Goal statement**
>
> Design, implement, test, integrate, and deploy a secure functional prototype that supports apartment administration, occupancy, billing, maintenance, facilities, visitors, and communications through coordinated microservices.

3. Primary Users and Roles

Role	Typical Responsibilities
System Administrator	Manage system-level roles, access settings, reference data, and audit visibility.
Apartment Manager	Oversee buildings, units, residents, occupancy, financial status, maintenance, facilities, and announcements.
Owner	View owned units, occupancy, charges, receipts, requests, and relevant notices.
Tenant / Resident	View profile and unit information, charges, payments, requests, facility bookings, visitor records, and announcements.
Finance Officer	Generate charges and invoices, record or verify simulated payments, review arrears, and produce summaries.
Maintenance Coordinator	Review requests, prioritize work, assign technicians, track progress, and close work orders.
Technician / Service Staff	View assigned work, update status, record actions, and report completion.
Security Officer	Record or validate visitors and view approved visitor information.

4. Team Domain Allocation

Group	Domain Ownership	Owned Microservices	Frontend Feature Area
Group 1	Identity, access, residents, and user relationships	identity-access-service; resident-management-service	Login, profile, role-aware navigation, resident/owner/tenant administration.
Group 2	Property, units, leases, and occupancy	property-unit-service; lease-occupancy-service	Building/unit administration, ownership, lease, occupancy, and availability views.
Group 3	Billing, charges, invoices, and payment records	billing-payment-service; utility-charge-service	Invoices, charges, payment recording, receipts, arrears, and financial summaries.
Group 4	Operations, maintenance, facilities, visitors, and communications	operations-service; community-service	Maintenance and complaints, facility booking, visitor records, announcements, and notifications.

The API Gateway, shared frontend shell, Docker Compose integration, and common release documentation are jointly governed by all four teams through the Project Integration Council.

5. Group 1 Scope - Identity and Resident Management

- Register and manage user accounts for administrators, managers, owners, tenants, finance staff, technicians, and security staff.
- Authenticate users and issue JWTs; support role-based access to endpoints and frontend routes.
- Create and maintain resident, owner, tenant, and staff profiles with status and contact information.
- Associate users with apartment units through validated service interactions rather than direct database access.
- Support account activation, deactivation, password change, and administrative role assignment.
- Provide audit-friendly metadata such as created time, updated time, and status history where appropriate.

Key business rule	A user may hold more than one permitted relationship, such as owner of one unit and resident of another. Access must be determined by role and verified unit relationship.

6. Group 2 Scope - Property, Lease, and Occupancy

- Create and manage buildings, floors, units, unit types, ownership records, and unit status.
- Maintain lease or occupancy agreements with start date, end date, status, occupants, and relevant notes.
- Prevent overlapping active occupancy records for the same unit unless a documented multi-occupancy rule applies.
- Provide unit availability, current occupant, owner relationship, and occupancy history views.
- Expose validated APIs for other services to confirm unit, owner, tenant, and occupancy status.
- Support status transitions such as available, reserved, occupied, under maintenance, and inactive.

Key business rule	A lease or occupancy record cannot become active unless the unit exists, the responsible parties are valid, and the requested period does not conflict with an active record.

7. Group 3 Scope - Billing, Utilities, and Payments

- Define recurring apartment charges such as management fee, facility fee, parking fee, and utility charge categories.
- Generate invoices for valid units or occupants using configurable charge rules and billing periods.
- Record simulated payments with amount, date, method, reference, status, and receipt information.
- Calculate outstanding balance, payment status, and arrears without using a real payment gateway.
- Maintain utility charge entries or summarized meter values where applicable.
- Provide resident and management views of invoices, receipts, payment history, and financial summaries.
- Prevent overpayment or invalid status transitions unless a documented adjustment or credit process is implemented.

Key business rule	Invoices must retain an auditable snapshot of charge values. Later changes to charge rules must not silently alter already-issued invoices.

8. Group 4 Scope - Operations and Community Services

- Allow residents or staff to submit maintenance requests or complaints with category, priority, description, attachment reference, and status.
- Allow coordinators to assign technicians, schedule work, update progress, record actions, and close requests with resolution details.
- Manage shared facilities and booking slots, including capacity, operating rules, availability, approval, cancellation, and conflict prevention.

- Record visitor requests or visits with resident/unit reference, date, purpose, status, and basic check-in information.
- Publish role-targeted announcements and maintain read or visibility status where feasible.
- Provide notifications within the application when important workflow events occur.

Key business rule	A facility booking cannot be approved when the facility is unavailable, the time overlaps another approved booking, or the requesting user lacks a valid apartment relationship.

9. Required Cross-Team Workflows

Workflow	Minimum Cross-Service Interaction
Resident onboarding	Identity/resident account is created; unit and occupancy are validated; authorized access becomes available in the shared frontend.
Monthly billing	Billing service confirms eligible unit or occupancy data, creates invoice, and displays it to the authorized resident or owner.
Payment recording	Finance user records a simulated payment; invoice balance changes; resident sees receipt and updated status.
Maintenance request	Authorized resident submits a request for a valid unit; coordinator assigns work; technician updates it; requester sees resolution.
Facility reservation	Resident relationship is validated; booking rules are checked; reservation is created and visible to management.
Visitor handling	Resident and unit are validated; visitor entry is recorded; authorized security or management users can view relevant information.

10. Core Entities

Domain	Indicative Entities
Identity and resident	User, Role, Permission, UserRole, ResidentProfile, OwnerProfile, StaffProfile, AccountStatus.
Property and occupancy	Building, Floor, Unit, UnitType, Ownership, Lease, Occupant, OccupancyStatus.
Billing and utilities	ChargeRule, Invoice, InvoiceLine, Payment, Receipt, UtilityCharge, Adjustment.
Operations and community	MaintenanceRequest, WorkOrder, TechnicianAssignment, Facility, Booking, Visitor, Announcement, Notification.

Teams must refine these entities and may combine or separate them when justified. The final design must preserve service data ownership and avoid a shared monolithic database.

11. System-Wide Non-Functional Requirements

- Security: protected operations require valid JWTs and appropriate roles; passwords must be securely hashed; sensitive fields must not be exposed unnecessarily.
- Usability: role-relevant navigation, clear validation messages, responsive layouts, and consistent design across modules.
- Reliability: failures from dependent services must produce understandable responses and must not corrupt local data.
- Performance: common list and detail operations should respond within a reasonable prototype target under normal classroom-scale load; teams must record observations.
- Maintainability: clear repository structure, consistent conventions, service README files, API documentation, and meaningful automated tests.

- Auditability: important status changes and financial operations should retain timestamps and responsible user information.
- Privacy: use synthetic data only; do not collect or publish real resident identity, contact, financial, or visitor information.

12. Minimum Reports and Dashboards

- Unit occupancy and availability summary.
- Outstanding invoice and arrears summary.
- Maintenance requests by status, priority, and category.
- Facility booking calendar or utilization summary.
- Recent announcements and operational alerts.
- Role-appropriate personal dashboard for resident, manager, finance officer, or technician.

13. Out of Scope

- Real banking or card payment gateway integration.
- Legal electronic signatures, lease certification, or regulatory compliance certification.
- Biometric access control, physical gate hardware, IoT meter integration, or smart-lock integration.
- Native mobile application.
- Real SMS or email delivery; in-application or simulated notifications are sufficient.
- Commercial warranty, production support, real resident onboarding, or live operational use.

14. Acceptance Conditions

[ ] All four team domains are accessible through one shared frontend and API Gateway.

[ ] At least two end-to-end workflows cross service or team boundaries and are demonstrated successfully.

[ ] Authentication, authorization, validation, error handling, data persistence, and logging are demonstrated.

[ ] Each owned service has Swagger/OpenAPI documentation, database scripts, automated tests, and a reproducible local setup.

[ ] The integrated release can be started locally using documented Docker Compose steps and is also deployed to an approved environment.

[ ] Jira, GitHub, testing, release, and project closure evidence is complete.